	INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO - Campus Bragança Paulista **Curso:** Análise e Desenvolvimento de Sistemas - ADS **Disciplina:** Qualidade de Software | Projeto Integrador: Sistema SaaS Defesa em Foco **Prof.** Wilson Vendramel **Nome:** Reinaldo Morais dos Santos (preencher/conferir) BP: BP303812X **Nome:** Geisiele (Integrante Defesa em Foco) BP: BP303XXXX **Nome:** Pedro (Integrante Defesa em Foco) BP: BP303XXXX **Nome:** Ramona (Integrante Defesa em Foco) BP: BP303XXXX

Trabalho 2 de Qualidade de Software

Instruções Gerais. Leia com bastante atenção.

a) Este trabalho contém uma lista de exercícios conceituais (conhecimento declarativo) e práticos (conhecimento funcional) de Qualidade e Testes de Software e corresponde a 50% do processo avaliativo da N2. Tal trabalho possui ao todo 4 grupos de exercícios e deve ser realizado em grupo de 3 a 5 estudantes;

b) Este documento deve ser entregue em um único arquivo PDF de forma organizada e legível. Além disto, os exercícios devem ser apresentados conforme enunciados e seções, caso contrário, o exercício correspondente será anulado. Se for favorecer a visualização do artefato, links de acesso a arquivos de imagem podem ser inseridos no presente documento. Ainda cabe frisar que os exercícios com artefatos ilegíveis serão anulados;

c) O trabalho em foco deve ser entregue como tarefa na plataforma Moodle até às 23h59 de 22/11/2026. A entrega do trabalho por um único integrante do grupo já é suficiente, assim sendo, deve ficar claro que não há a necessidade de todos os integrantes da equipe entregarem tal tarefa;

d) As dúvidas enviadas via e-mail institucional podem ser respondidas uma vez que sejam submetidas até às 19h00 de 19/11/2026;

e) O trabalho deve ser verificado e validado pelo próprio grupo antes de ser entregue, dado que não serão aceitas novas entregas, parcial ou totalmente após o prazo final;

f) Os materiais disponibilizados auxiliam a realização dos exercícios deste trabalho;

g) Os trabalhos com respostas suspeitas de plágio serão devidamente anulados e zerados. Cada exercício que apresentar a mesma resposta em dois ou mais trabalhos será devidamente anulado e a respectiva pontuação descontada;

h) Exceções sobre as instruções supracitadas devem ser tratadas com o próprio professor antecipadamente.

Grupo 1: Especificação e Arquitetura do Software (2,0 pontos)

Antes de iniciar os exercícios, cabe frisar que a seleção do caso de uso para teste deve seguir os critérios abaixo:

a) Relevância do caso de uso para o domínio de negócio, ou seja, deve ser selecionado um caso de uso principal (core) do sistema;
b) Presença de fluxos alternativos e/ou de exceção e de, no mínimo, três regras de negócio relacionadas ao caso de uso;
c) Implementação do caso de uso.

1- Reproduza, no Quadro 1a, a descrição textual do principal caso de uso do sistema extraído da especificação do sistema de software selecionado para o presente Plano de Teste. Atenção aos fluxos alternativos e de exceção, assim como, ao mínimo de três regras de negócio vinculadas.

Quadro 1a – Descrição Textual do Caso de Uso: UC01 – Registrar e Gerenciar Ocorrência de Defesa Civil

Caso de Uso: UC01 – Registrar e Gerenciar Ocorrência de Desastre / Emergência (Core do Sistema Defesa em Foco)
Descrição: Permite que cidadãos, agentes operacionais e coordenadores da Defesa Civil registrem ocorrências geolocalizadas de desastres naturais ou incidentes urbanos (alagamentos, deslizamentos, quedas de árvore, incêndios em vegetação, colapsos estruturais), realizando validação antifraude de raio GPS (Haversine), clusterização espacial automática via PostGIS, classificação nacional COBRADE, armazenamento de evidências fotográficas em nuvem S3 (MinIO), sugestão de posicionamento assistida por Inteligência Artificial (Google Gemini) e controle completo do ciclo de vida do atendimento.
Ator Primário: Cidadão (usuário móvel/anônimo ou autenticado), Agente Operacional de Defesa Civil e Coordenador Municipal (Administrador).
Ator(es) Secundário(s): 1) Banco Geoespacial PostGIS (cálculo de centroide ST_DWithin); 2) MinIO Object Storage (armazenamento S3 de fotos Base64); 3) OneSignal Push API (disparo de alertas push aos coordenadores e cidadãos); 4) Google Gemini 1.5 Flash API (auditoria multimodal de imagem e sugestão de coordenadas).
Precondições: 1) O aplicativo móvel (Flutter) ou Web Dashboard deve possuir conectividade com a API REST (ou operar em fila local Hive para sincronização posterior); 2) Para operações de auto-aprovação, chegada em campo ou resolução, o usuário deve possuir token JWT válido no contexto de segurança (Spring Security) com perfil AGENTE, ADMINISTRADOR ou SUPER_ADMIN.
Fluxo Principal (Registro por Cidadão e Ciclo de Atendimento): **1) [Estímulo]** O Ator Primário (Cidadão) seleciona a opção 'Registrar Ocorrência', informa a natureza do desastre (tipo), redige a descrição, confirma a coordenada no mapa (latitude/longitude), anexa opcionalmente uma foto e submete o formulário (POST /api/ocorrencias). **2) [Resposta]** O Sistema (OcorrenciaService) valida o preenchimento dos campos obrigatórios (tipo e descrição), sanitiza o texto contra injeção HTML/XSS e calcula a distância geodésica (fórmula de Haversine) entre o GPS real do envio e o ponto reportado no mapa (RN01). **3) [Resposta]** O Sistema consulta a extensão espacial PostGIS (buscarCentroideProximo) para identificar relatos ativos da mesma natureza em um raio de 200 metros nas últimas 2 horas, incrementando o contador comunitário totalRelatosCluster (RN02). **4) [Resposta]** O Sistema normaliza o identificador do município, vincula a entidade Cidade e classifica automaticamente o evento com o código e descrição oficiais do protocolo federal COBRADE/S2ID (RN04). **5) [Resposta]** Por se tratar de relato de Cidadão comum, o Sistema define o status inicial como PENDENTE_APROVACAO, persiste o registro no PostgreSQL e aciona o OneSignal para enviar notificações Push a todos os Coordenadores (ADMINISTRADOR) daquele município (RN03). **6) [Estímulo]** O Coordenador Municipal acessa a ocorrência pendente no Web Dashboard, solicita opcionalmente a sugestão analítica de posicionamento (GET /api/ocorrencias/{id}/sugestao-ia) e aciona a aprovação da ocorrência (PUT /api/ocorrencias/{id}/aprovar) (RN06). **7) [Resposta]** O Sistema altera o status da ocorrência para APROVADA, publica o ponto no mapa público da cidade e dispara uma notificação Push ao Cidadão autor informando que seu relato foi verificado e publicado. **8) [Estímulo]** O Agente Operacional desloca-se até o local do incidente e registra sua chegada pelo aplicativo

móvel informando o parecer preliminar da situação (PUT /api/ocorrencias/{id}/chegada).

9) [Resposta] O Sistema marca a flag agenteNoLocal = true, registra o timestamp exato em dataChegadaAgente e transiciona o status da ocorrência para TRABALHANDO_ATUALMENTE.

10) [Estímulo / Resposta] Após conter o evento, o Agente/Coordenador informa o parecer técnico conclusivo e encerra o atendimento (PUT /api/ocorrencias/{id}/resolver). O Sistema atualiza o status para RESOLVIDA, grava dataResolucao e notifica o Cidadão autor.

Fluxos Alternativos:

FA01 – Auto-aprovação para Agente Operacional ou Coordenador (Passo 5): a) Caso o usuário autenticado possua perfil AGENTE, ADMINISTRADOR ou SUPER_ADMIN (ou flag criadoPorAgente = true) e a ocorrência não apresente origem suspeita (distância GPS <= 2.000m), o Sistema força a vinculação ao município de jurisdição do agente (RN05) e define o status diretamente como APROVADA, dispensando fila de moderação.

FA02 – Registro Retroativo de Ocorrência Passada pelo Coordenador (Passo 5): a) Se o registro for submetido por usuário privilegiado (autoAprovado = true) com dataHora informada no passado (anterior a 2 minutos do horário atual do servidor), o Sistema reconhece como lançamento histórico/retroativo, definindo automaticamente status = RESOLVIDA e preenchendo dataResolucao (RN03).

FA03 – Upload de Evidência Fotográfica em Base64 para o MinIO (Passo 4): a) Se o campo caminhoFoto contiver um payload codificado no padrão 'data:image/...;base64', o Sistema aciona MinioService.uploadBase64Image(), armazena o arquivo binário no bucket privado 'ocorrencias' e gera uma URL pré-assinada temporária na resposta.

FA04 – Reativação de Ocorrência Resolvida (Pós-Passo 10): a) Caso um foco de desastre volte a apresentar risco após encerrado, o Agente ou Coordenador aciona PUT /api/ocorrencias/{id}/reativar; o Sistema retorna o status de RESOLVIDA para APROVADA e limpa o campo dataResolucao.

Fluxos de Exceção:

FE01 – Dados Obrigatórios Ausentes ou Em Branco (Passo 2): a) Se o atributo 'tipo' ou 'descricao' for nulo ou composto apenas por espaços em branco, o método registrarOcorrencia() interrompe imediatamente a execução e lança IllegalArgumentException ('Tipo da ocorrência é obrigatório' / 'Descrição é obrigatória'), retornando HTTP 400 Bad Request.

FE02 – Detecção de Origem Suspeita / Possível Trote por Distância GPS > 2.000m (Passo 2): a) Se as coordenadas do GPS físico do aparelho no momento do envio (latitudeEnvio/longitudeEnvio) distarem mais de 2.000 metros do ponto marcado no mapa, o Sistema define origemSuspeita = true, emite alerta em log e bloqueia qualquer auto-aprovação, retendo obrigatoriamente a ocorrência em PENDENTE_APROVACAO para auditoria manual (RN01).

FE03 – Violação de Jurisdição Municipal por Agente ou Coordenador (Passos 6 a 10): a) Se um usuário com perfil AGENTE ou ADMINISTRADOR tentar aprovar, registrar chegada, editar, resolver ou excluir uma ocorrência pertencente a outro município diferente de sua lotação cadastral, o método checkJurisdiction() lança SecurityException ('Acesso negado: Você só pode gerenciar itens de sua própria cidade'), retornando HTTP 403 Forbidden (RN05).

Pós-condições: 1) A ocorrência encontra-se devidamente persistida na tabela 'ocorrencias' com protocolo UUID, classificação COBRADE válida e contagem de cluster atualizada; 2) Fotos anexadas estão salvas de forma segura no bucket MinIO e acessíveis via Presigned URL; 3) O status reflete fielmente a etapa do ciclo operacional e as partes interessadas receberam notificação Push via OneSignal.

Regras de Negócio Relacionadas (implementadas em OcorrenciaService, OcorrencialaService e CobradeUtil):

RN01 (Validação Geodésica Antifraude – Raio de 2 km): Sempre que o dispositivo enviar as coordenadas reais do GPS de envio (latitudeEnvio, longitudeEnvio), o sistema deve calcular a distância geodésica de Haversine até a coordenada reportada. Caso a distância seja estritamente superior a 2.000 metros (distMetros > 2000.0), a ocorrência deve ser marcada com origemSuspeita = true e ter sua auto-aprovação bloqueada.

RN02 (Validação Comunitária e Clusterização Espacial PostGIS): No ato do registro, o sistema deve consultar no PostGIS (ST_DWithin) todas as ocorrências não recusadas e não resolvidas do mesmo 'tipo' registradas nas últimas 2 horas dentro de um raio de 200 metros, atribuindo totalRelatosCluster = totalAnteriores + 1 (ou 1 em caso de relato isolado ou falha de banco).

RN03 (Política de Status Inicial, Auto-Aprovação e Registro Retroativo): Relatos enviados por cidadãos comuns ou marcados com origemSuspeita = true iniciam obrigatoriamente no status PENDENTE_APROVACAO e disparam alerta Push aos administradores da cidade. Relatos criados por

usuários autenticados com perfil AGENTE, ADMINISTRADOR ou SUPER_ADMIN (sem origem suspeita) recebem auto-aprovação: se a dataHora informada for retroativa (anterior a 2 minutos do instante atual), entram diretamente como RESOLVIDA; caso contrário, entram como APROVADA.

RN04 (Padronização Automática pelo Protocolo Federal COBRADE): Toda ocorrência deve possuir código e descrição oficiais da Classificação e Codificação Brasileira de Desastres (MDR/S2ID). Quando não informado manualmente no payload, o sistema deve resolvê-lo automaticamente a partir do tipo do evento (ex.: alagamento -> 1.2.3.0.0; deslizamento -> 1.3.2.1.1; queda_arvore -> 1.3.1.1.1; incendio_vegetacao -> 1.4.1.1.0; default -> 9.9.9.9.9).

RN05 (Isolamento Multi-Tenant e Jurisdição Municipal Estrita): Usuários com papel AGENTE ou ADMINISTRADOR estão restritos exclusivamente à jurisdição do município vinculado ao seu cadastro: ao criarem ocorrências, o sistema força o código da cidade do próprio agente; ao tentarem aprovar, atender, alterar ou remover ocorrências de outro município, a operação é bloqueada com SecurityException. Apenas o perfil SUPER_ADMIN possui visibilidade e gerência irrestrita entre municípios.

RN06 (Elegibilidade de Inteligência Artificial Multimodal por Plano SaaS): A auditoria multimodal de fotos e sugestão de coordenadas via Google Gemini 1.5 Flash (OcorrenciaIaService) é exclusiva para municípios assinantes do plano PRO_MUNICIPAL; municípios nos planos BASE_GRATUITO ou GESTAO_MUNICIPAL (ou em caso de timeout/HTTP 429 da API Gemini) recebem automaticamente o fallback seguro baseado no centroide geométrico do PostGIS.

2- Modele um Diagrama de Classes representando a lógica do caso de uso escolhido em uma camada (pacote) de aplicação e as classes de entidade em uma camada (pacote) de domínio. Tal diagrama de classes deve representar os detalhes das classes *use case/service*, especificamente métodos e seus parâmetros, e os detalhes das classes de entidade, especificamente atributos e seus tipos. Os relacionamentos devem exibir navegabilidade e dependência.

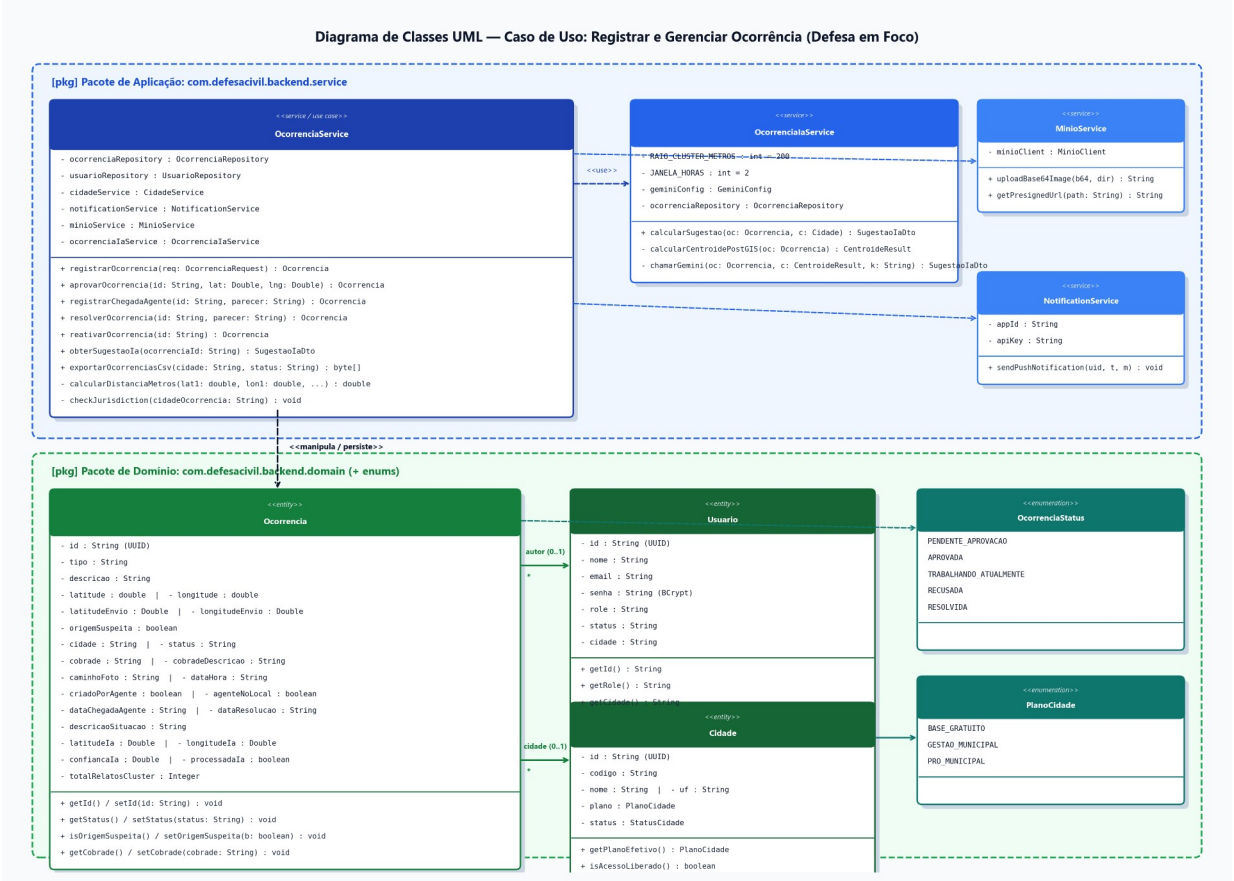


Figura 1 – Diagrama de Classes UML representando a camada de aplicação (pacote service) e a camada de domínio (pacote domain) do Caso de Uso UC01.

Observação Técnica: O diagrama evidencia a navegabilidade unidirecional da entidade Ocorrencia para Usuario (autor) e Cidade (cidadeEntidade), bem como as relações de dependência (<<use>> / <<manipula>>) de OcorrenciaService para os serviços auxiliares e repositórios.

3- Represente o Diagrama Arquitetural do Sistema de Software com notações visuais livres para representar os componentes arquiteturais (front-end, back-end, persistência, APIs, serviços de nuvem…). As tecnologias sejam identificadas junto aos componentes correspondentes.

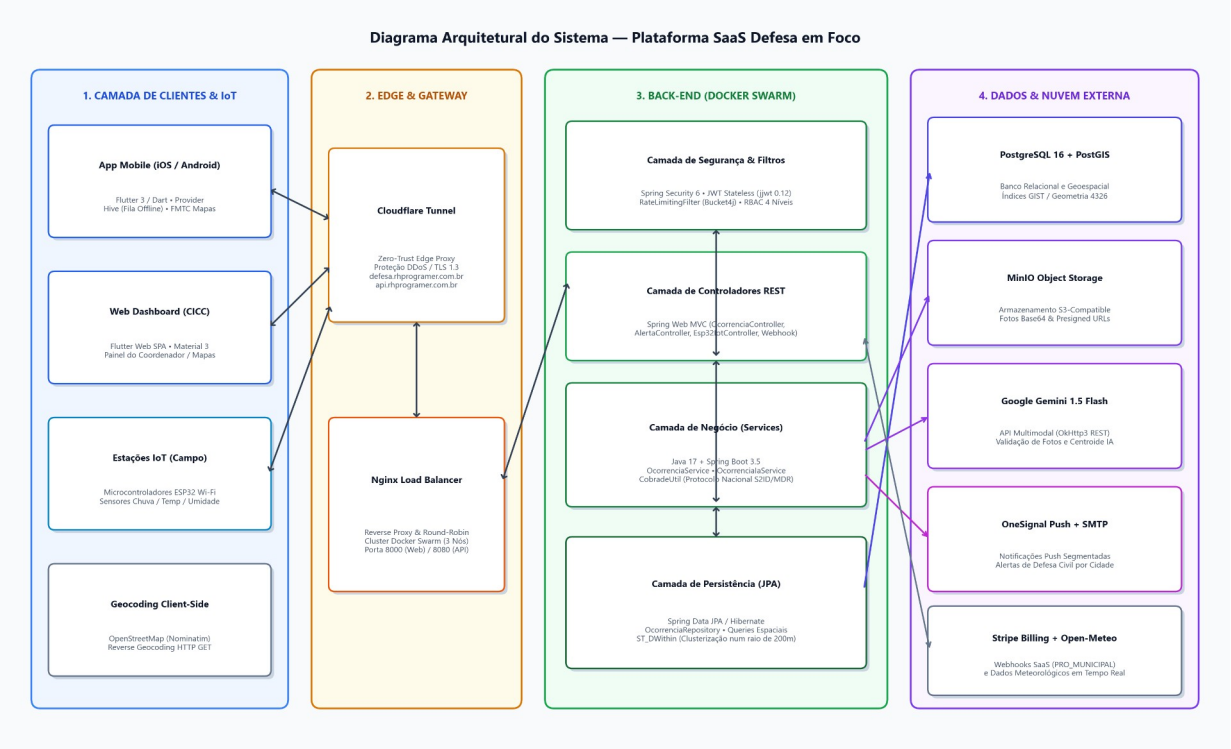


Figura 2 – Diagrama Arquitetural completo da plataforma Defesa em Foco com identificação das tecnologias por componente.
Resumo Arquitetural: Arquitetura distribuída em 4 camadas operando em cluster Docker Swarm de 3 nós (Mac mini On-Premises + Cloudflare Zero-Trust Tunnel + Nginx Load Balancer), integrando Front-end Flutter (Mobile/Web com persistência offline Hive), Back-end Spring Boot 3 / Java 17, Banco PostgreSQL 16 + PostGIS, Object Storage MinIO S3 e APIs externas (Google Gemini 1.5 Flash, OneSignal, Stripe e Open-Meteo).

Grupo 2: Projeto dos Casos de Teste, Execução dos Testes de Sistema e Análise dos Resultados (3,0 pontos)

4- Represente, em um Diagrama de Atividades, a lógica do caso de uso selecionado, contemplando seus fluxos principal, alternativos e de exceção. Organize as atividades em raias (*swimlanes*) distintas: uma para o ator primário, outra para o sistema em foco e, quando aplicável, outras para os atores secundários (sistemas externos). Atenção às regras de negócio vinculadas ao caso de uso em questão. Visando aplicar Teste Baseado em Modelo, mapeie o Diagrama de Atividades para um Grafo de Fluxo de Controle (GFC), adaptando assim o teste de caminho da técnica estrutural para o teste de cenário da técnica funcional e buscando gerar os casos de testes para os cenários (fluxos) do caso de uso. Preencha o Quadro 2a para percorrer cada caminho independente do GFC ao menos uma vez, salientando que novas linhas podem ser adicionadas.

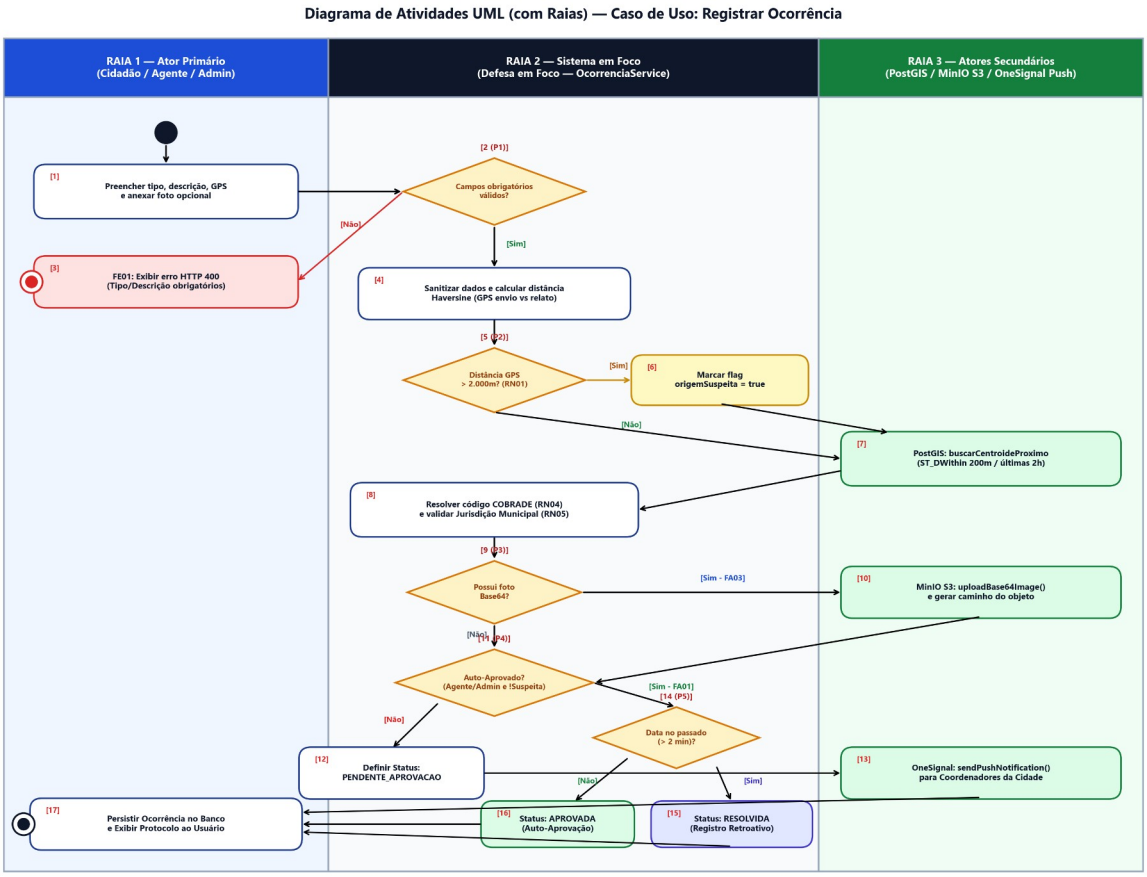


Figura 3 – Diagrama de Atividades UML em 3 raias (Ator Primário, Sistema em Foco e Atores Secundários) contemplando fluxos principal, alternativos, exceções e regras de negócio.

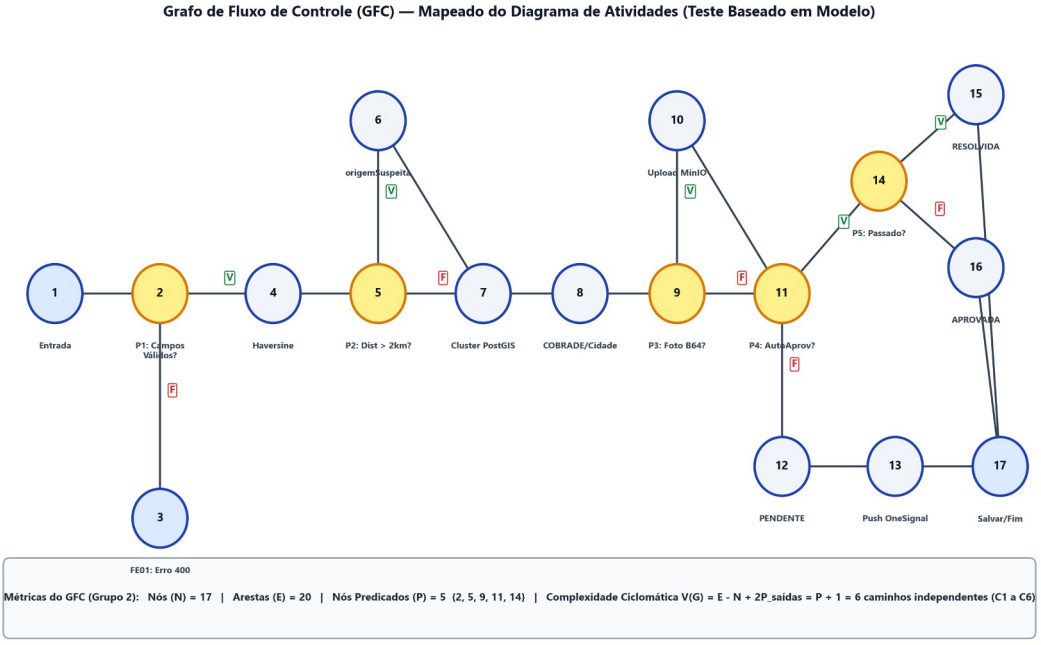


Figura 4 – Grafo de Fluxo de Controle (GFC) mapeado a partir do Diagrama de Atividades para aplicação de Teste Baseado em Modelo (V(G) = P + 1 = 6 caminhos independentes).

Quadro 2a: Caminhos independentes do GFC

#Caminho	Caminho	Nós Predicados
C1	1 → 2 → 3 (Fim de Exceção FE01)	Nó 2 (P1 = Falso: tipo ou descrição nulos/em branco → HTTP 400)

C2	1 → 2 → 4 → 5 → 7 → 8 → 9 → 11 → 12 → 13 → 17	Nó 2 (V), Nó 5 (P2 = F: dist <= 2km), Nó 9 (P3 = F: sem foto), Nó 11 (P4 = F: Cidadão → PENDENTE_APROVACAO + Push)
C3	1 → 2 → 4 → 5 → 6 → 7 → 8 → 9 → 11 → 12 → 13 → 17	Nó 2 (V), Nó 5 (P2 = V: dist > 2km → origemSuspeita=true), Nó 9 (F), Nó 11 (P4 = F: bloqueio antifraude RN01)
C4	1 → 2 → 4 → 5 → 7 → 8 → 9 → 10 → 11 → 14 → 16 → 17	Nó 2 (V), Nó 5 (F), Nó 9 (P3 = V: upload foto Base64 MinIO), Nó 11 (P4 = V: Agente/Admin), Nó 14 (P5 = F: tempo real → APROVADA)
C5	1 → 2 → 4 → 5 → 7 → 8 → 9 → 11 → 14 → 15 → 17	Nó 2 (V), Nó 5 (F), Nó 9 (F), Nó 11 (P4 = V: Agente/Admin), Nó 14 (P5 = V: data retroativa > 2min → RESOLVIDA)
C6	1 → 2 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 → 13 → 17	Nó 2 (V), Nó 5 (P2 = V: origemSuspeita), Nó 9 (P3 = V: com foto MinIO), Nó 11 (P4 = F: retido para moderação)

5- Modele um Diagrama de Transição de Estados para representar, conforme a sequência dos eventos, os estados e transições de um objeto (classe) relevante em termos de mudança de estados, proveniente do caso de uso descrito a priori. Atenção com as regras de negócio vinculadas ao caso de uso em questão. Visando aplicar Teste Baseado em Modelo, mapeie o Diagrama de Transição de Estados para um Grafo de Estados (GE), buscando gerar os casos de teste com base nas transições e estados do objeto (classe) do caso de uso. Preencha o Quadro 2b para percorrer cada sequência de transições independente do GE ao menos uma vez, salientando que novas linhas podem ser adicionadas.

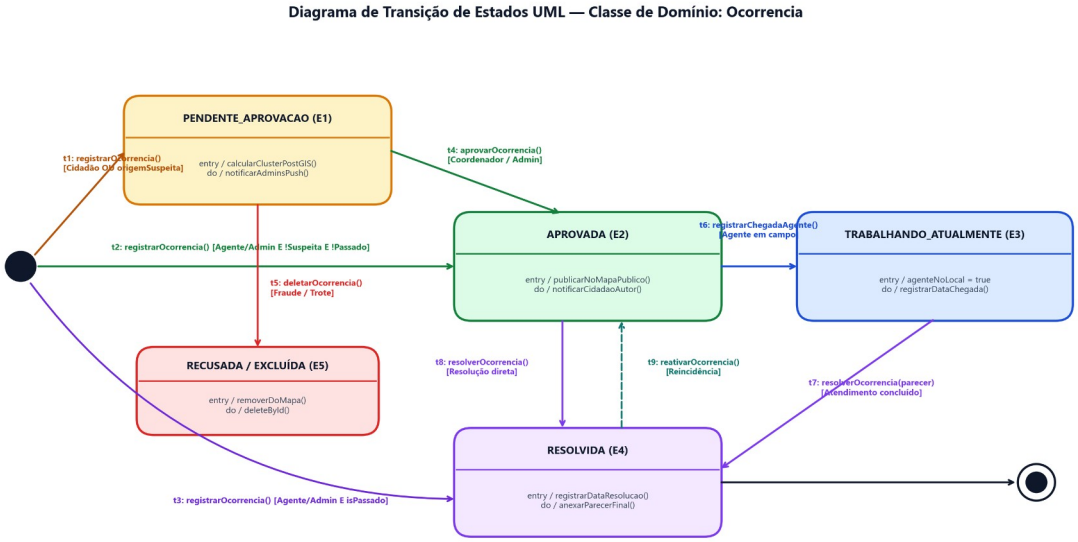


Figura 5 – Diagrama de Transição de Estados UML do objeto Ocorrencia ao longo do seu ciclo de vida operacional.

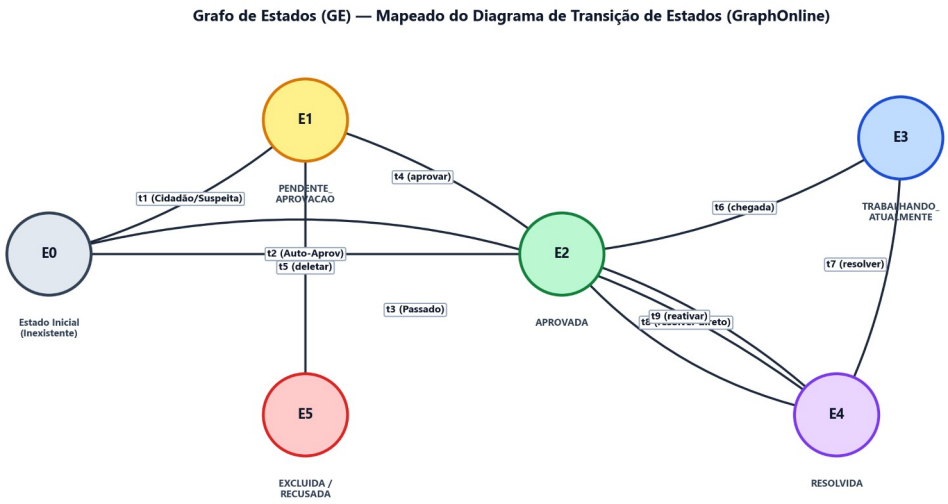


Figura 6 – Grafo de Estados (GE) mapeado para derivação das sequências de transições independentes (E0 a E5, transições t1 a t9).

Quadro 2b: Sequências independentes do GE

#Sequência	Sequência de transições	Estados envolvidos
S1	t1 (registrarOcorrencia[Cidadão]) → t4 (aprovarOcorrencia) → t6 (registrarChegadaAgente) → t7 (resolverOcorrencia)	E0 (Inicial) → E1 (PENDENTE_APROVACAO) → E2 (APROVADA) → E3 (TRABALHANDO_ATUALMENTE) → E4 (RESOLVIDA)
S2	t1 (registrarOcorrencia[Suspeita/Trote]) → t5 (deletarOcorrencia)	E0 (Inicial) → E1 (PENDENTE_APROVACAO) → E5 (EXCLUÍDA / RECUSADA)
S3	t2 (registrarOcorrencia[Agente/Admin Tempo Real]) → t6 (registrarChegadaAgente) → t7 (resolverOcorrencia) → t9 (reativarOcorrencia)	E0 (Inicial) → E2 (APROVADA) → E3 (TRABALHANDO_ATUALMENTE) → E4 (RESOLVIDA) → E2 (APROVADA)
S4	t2 (registrarOcorrencia[Agente/Admin]) → t8 (resolverOcorrencia[Direto pelo Coordenador])	E0 (Inicial) → E2 (APROVADA) → E4 (RESOLVIDA)
S5	t3 (registrarOcorrencia[Admin Data Passada > 2min])	E0 (Inicial) → E4 (RESOLVIDA - Lançamento Histórico Retroativo)

6- Utilizando a planilha de testes disponibilizada na plataforma Teams, projete os casos de teste considerando os cenários do caso de uso e as técnicas de teste funcional: Partição de Equivalência, Análise de Valor Limite, Teste Baseado em Modelo e Teste de Caso de Uso. Para cada condição de entrada, identifique claramente a regra de negócio correspondente. Cabe frisar que novas colunas de condição de entrada podem ser adicionadas na planilha.

7- Execute os testes de sistema para tal caso de uso e registre com clareza os resultados, levando em consideração cada caso de teste previamente projetado. Cabe ressaltar que cada teste, revelando defeito ou não, deve ser evidenciado separadamente via vídeo e o respectivo link de acesso precisa constar na planilha. Cabe lembrar que a técnica de teste a ser aplicada aqui é funcional de abordagem caixa-preta.

Link de Acesso à Planilha de Testes de Sistema (Exercícios 6 e 7):
https://github.com/moraisReinaldo/Defesa/blob/master/Planilha_Testes_Sistema_QSW_Defesa_em_Foco.xlsx (Arquivo também gerado localmente em D:\GitHub\Defesa\Planilha_Testes_Sistema_QSW_Defesa_em_Foco.xlsx).
Resumo Executivo dos Casos de Teste Funcionais (Caixa-Preta) Projetados e Executados na Planilha:
• **CT-SIS-01 [Partição de Equivalência - Inválida | RN04]:** Entrada: tipo = '' (vazio), descricao = 'Alagamento na via'. Resultado Esperado e Obtido: HTTP 400 Bad Request ('Tipo da ocorrência é obrigatório'). Status: PASSOU.
• **CT-SIS-02 [Partição de Equivalência - Inválida | RN04]:** Entrada: tipo = 'alagamento', descricao = ' ' (em branco). Resultado Esperado e Obtido: HTTP 400 Bad Request ('Descrição é obrigatória'). Status: PASSOU.
• **CT-SIS-03 [Análise de Valor Limite - Fronteira Inferior <= 2.000m | RN01]:** Entrada: Agente registra ocorrência com distância GPS = 1.995m (<= 2.000m). Resultado Obtido: origemSuspeita = false; auto-aprovação concedida (status =

APROVADA). Status: PASSOU.
• **CT-SIS-04 [Análise de Valor Limite - Fronteira Superior > 2.000m | RN01, RN03]:** Entrada: Agente registra ocorrência com distância GPS = 2.005m (> 2.000m). Resultado Obtido: origemSuspeita = true; auto-aprovação bloqueada (status = PENDENTE_APROVACAO). Status: PASSOU.
• **CT-SIS-05 [Análise de Valor Limite - Raio Cluster PostGIS <= 200m / 2h | RN02]:** Entrada: Novo relato de 'alagamento' a 150m de 2 relatos ativos nas últimas 2h. Resultado Obtido: totalRelatosCluster = 3 calculado via ST_DWithin. Status: PASSOU.
• **CT-SIS-06 [Análise de Valor Limite - Janela Temporal Retroativa > 2 min | RN03]:** Entrada: Administrador registra ocorrência com dataHora = agora - 5 minutos. Resultado Obtido: isPassado = true; status inicial = RESOLVIDA com dataResolucao preenchida. Status: PASSOU.
• **CT-SIS-07 [Teste Baseado em Modelo - GFC Caminho C2 e GE Sequência S1 | RN03, RN04]:** Entrada: Cidadão registra 'deslizamento' -> Admin aprova -> Agente informa chegada -> Agente resolve. Resultado Obtido: COBRADE = 1.3.2.1.1; transições E1 -> E2 -> E3 -> E4 concluídas e Push enviado. Status: PASSOU.
• **CT-SIS-08 [Teste Baseado em Modelo - GE Sequência S3 Reativação | FA04]:** Entrada: Coordenador aciona reativação de ocorrência RESOLVIDA. Resultado Obtido: status retorna para APROVADA e dataResolucao = null. Status: PASSOU.
• **CT-SIS-09 [Partição de Equivalência - Jurisdição Inválida | RN05, FE03]:** Entrada: Agente lotado em 'sao_paulo' tenta resolver ocorrência de 'joanopolis'. Resultado Obtido: HTTP 403 Forbidden (SecurityException). Status: PASSOU.
• **CT-SIS-10 [Teste de Caso de Uso - Sugestão IA Plano PRO vs Base | RN06]:** Entrada: Coordenador solicita sugestão IA em município PRO_MUNICIPAL (Gemini ativo) vs BASE_GRATUITO (Fallback PostGIS). Resultado Obtido: SugestaoIaDto retornado com score de confiança e centroide validado. Status: PASSOU.

Grupo 3: Implementação, Execução dos Testes de Unidade e Análise dos Resultados (3,0 pontos)

8- Mediante utilização da ferramenta ComplexGraph, gere o Grafo de Fluxo de Controle para o método mais complexo do caso de uso, considerando as classes *use case/service*. Analise o GFC e responda a quantidade de regiões, a complexidade ciclomática por meio da fórmula matemática e via lógica dos predicados. (https://code.google.com/archive/p/complexgraph/downloads)

V(G) = Regiões = 15 regiões (14 regiões fechadas delimitadas pelas arestas de decisão/loop + 1 região externa planar)
V(G) = E – N + 2 = 41 – 28 + 2 = 15 (onde E = 41 arestas de fluxo de controle e N = 28 nós/blocos básicos)
V(G) = P + 1 = 14 + 1 = 15 (onde P = 14 nós predicados de decisão binária/repetição identificados no método)

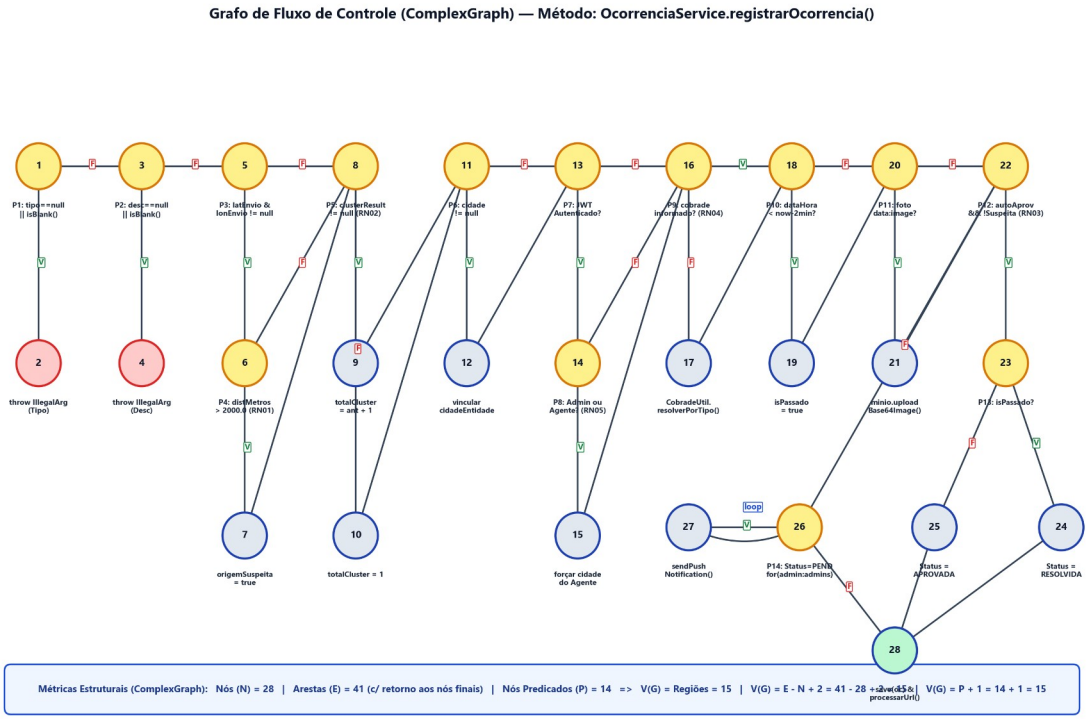


Figura 7 – Grafo de Fluxo de Controle (ComplexGraph) do método mais complexo do caso de uso: OcorrenciaService.registrarOcorrencia(OcorrenciaRequest).

Análise Estrutural Detalhada do Método OcorrenciaService.registrarOcorrencia(OcorrenciaRequest request):
• **Classe / Pacote:** com.defesacivil.backend.service.OcorrenciaService (linhas 119 a 271).
• **Justificativa da Escolha:** Trata-se do método de maior densidade algorítmica e decisória do Caso de Uso UC01, concentrando validações de entrada, cálculo trigonométrico espacial (Haversine), integração geoespacial PostGIS, autenticação JWT e controle de jurisdição RBAC, mapeamento nacional COBRADE, armazenamento S3 MinIO, regras de auto-aprovação, detecção de lançamento retroativo e disparo de notificações Push.
• **Mapeamento Completo dos 14 Nós Predicados (P = 14) no Código-Fonte:**
- P1 (Nó 1 - Linha 120): if (request.getTipo() == null || request.getTipo().isBlank()) → [V] Nó 2 (throw IllegalArgumentException) | [F] Nó 3.
- P2 (Nó 3 - Linha 123): if (request.getDescricao() == null || request.getDescricao().isBlank()) → [V] Nó 4 (throw IllegalArgumentException) | [F] Nó 5.
- P3 (Nó 5 - Linha 136): if (request.getLatitudeEnvio() != null && request.getLongitudeEnvio() != null) → [V] Nó 6 (calcula Haversine) | [F] Nó 8.
- P4 (Nó 6 - Linha 141 - RN01): if (distMetros > 2000.0) → [V] Nó 7 (oc.setOrigemSuspeita(true)) | [F] Nó 8.
- P5 (Nó 8 - Linha 152 - RN02): if (clusterResult != null && clusterResult.length == 3) → [V] Nó 9 (totalRelatosCluster = ant + 1) | [F] Nó 10 (totalRelatosCluster = 1).
- P6 (Nó 11 - Linha 165): if (cidade != null) → [V] Nó 12 (busca e vincula CidadeEntidade) | [F] Nó 13.
- P7 (Nó 13 - Linha 172): if (emailAutenticado != null && !'anonymousUser'.equals(emailAutenticado)) → [V] Nó 14 | [F] Nó 16 (usuário anônimo).
- P8 (Nó 14 - Linha 177 - RN05): if (Role.ADMINISTRADOR.equals(role) || Role.AGENTE.equals(role)) → [V] Nó 15 (força cidade de jurisdição do agente) | [F] Nó 16.
- P9 (Nó 16 - Linha 201 - RN04): if (request.getCobrade() != null && !request.getCobrade().isBlank()) → [V] usa COBRADE informado | [F] Nó 17 (CobradeUtil.resolverPorTipo()).
- P10 (Nó 18 - Linha 214/219): if (request.getDataHora() != null && dt.isBefore(now.minusMinutes(2))) → [V] Nó 19 (isPassado = true) | [F] Nó 20.
- P11 (Nó 20 - Linha 229): if (foto != null && foto.startsWith('data:image')) → [V] Nó 21 (minioService.uploadBase64Image()) | [F] Nó 22.

- **P12 (Nó 22 - Linha 236/247 - RN03):** if (autoAprovado) [(criadoPorAgente || Admin/Agente/SuperAdmin) && !origemSuspeita] → [V] Nó 23 | [F] Nó 26.
- **P13 (Nó 23 - Linha 248 - RN03):** if (isPassado) → [V] Nó 24 (status = RESOLVIDA) | [F] Nó 25 (status = APROVADA).
- **P14 (Nó 26 - Linha 261 - RN03):** for (Usuario admin : admins) → [V] Nó 27 (notificationService.sendPushNotification()) | [F] Nó 28 (save & return).

• **Conclusão da Análise Ciclomática:** As três abordagens matemáticas convergem exatamente para V(G) = 15, indicando que o conjunto básico de teste estrutural exige 15 caminhos linearmente independentes para cobrir 100% das arestas e decisões lógicas do método.

9- Implemente os testes de unidade com base nas classes da camada de aplicação (*use case/service*), nas classes de entidade da camada de domínio e nas regras de negócio relacionadas ao caso de uso selecionado. Sendo preciso, mocks, drivers e stubs podem ser adotados, contudo, deixe claro o uso desses recursos (via comentários no próprio código) para apoiar a compreensão desta fase de teste. Para o grupo que implementou o sistema de software em Java, se recomenda a utilização da ferramenta JUnit para se implementar tais testes; para o grupo que adotou outra tecnologia de desenvolvimento, uma ferramenta correspondente e apropriada deve ser utilizada. Vale ressaltar que o código referente à implementação dos testes deve ser apresentado de forma legível e organizado, por pacote e classe.

Link de Acesso ao Código dos Testes de Unidade no GitHub:
• Classe de Teste da Camada de Aplicação (Service):
https://github.com/moraisReinaldo/Defesa/blob/master/defesa-backend/src/test/java/com/defesacivil/backend/service/OcorrenciaServiceTest.java
• Classe de Teste da Camada de Domínio (Entity/COBRADE):
https://github.com/moraisReinaldo/Defesa/blob/master/defesa-backend/src/test/java/com/defesacivil/backend/domain/OcorrenciaTest.java

Listagem Organizada por Pacote e Classe (JUnit 5 + Mockito com indicação explícita de Mocks e Stubs):

```
// ================================================================================================
// PACOTE 1: com.defesacivil.backend.service  |  CLASSE: OcorrenciaServiceTest.java
// ================================================================================================
@ExtendWith(MockitoExtension.class)
class OcorrenciaServiceTest {
    // MOCKS E STUBS (Mockito): Isolam o SUT (OcorrenciaService) de infraestrutura externa
    @Mock private OcorrenciaRepository ocorrenciaRepository; // Mock/Stub BD PostgreSQL + PostGIS
    @Mock private UsuarioRepository usuarioRepository;       // Stub consulta de usuários/admins
    @Mock private CidadeService cidadeService;               // Stub normalização municipal
    @Mock private NotificationService notificationService;   // Mock verificação de envio Push OneSignal
    @Mock private MinioService minioService;                 // Stub upload Base64 e Presigned URL S3
    @Mock private OcorrenciaIaService ocorrenciaIaService;   // Stub cálculo de centroide e IA Gemini
    @InjectMocks private OcorrenciaService ocorrenciaService; // SUT (System Under Test)

    @Test // CT01 (FE01 / C1): Validação de tipo obrigatório
    void deveLancarExcecaoQuandoTipoObrigatorioAusente() {
        OcorrenciaRequest req = new OcorrenciaRequest();
        req.setTipo("   "); req.setDescricao("Alagamento severo");
        assertThrows(IllegalArgumentException.class, () -> ocorrenciaService.registrarOcorrencia(req));
        verify(ocorrenciaRepository, never()).save(any());
    }

    @Test // CT03 (RN01, RN02, RN03, RN04): Cidadão registra ocorrência <= 2km -> PENDENTE + Push
    void deveRegistrarOcorrenciaDeCidadaoComoPendenteENotificarAdmins() {
        OcorrenciaRequest req = criarRequestPadrao("alagamento", -22.9300, -46.2700, -22.9305, -46.2705);
        // Stub do PostGIS: simula 2 relatos anteriores num raio de 200m nas últimas 2h
        when(ocorrenciaRepository.buscarCentroideProximo(anyString(), anyDouble(), anyDouble(), eq(200), eq(2)))
            .thenReturn(new Object[]{2, -22.9302, -46.2702});
        when(usuarioRepository.findByCidadeAndRole("joanopolis", "ADMINISTRADOR"))
            .thenReturn(List.of(criarAdminMock("admin-01")));
        when(ocorrenciaRepository.save(any())).thenAnswer(i -> i.getArgument(0));

        Ocorrencia res = ocorrenciaService.registrarOcorrencia(req);
        assertEquals("PENDENTE_APROVACAO", res.getStatus());
        assertEquals("1.2.3.0.0", res.getCobrade()); // RN04: COBRADE resolvido automaticamente
        verify(notificationService, times(1)).sendPushNotification(eq("admin-01"), anyString(), anyString());
    }

    @Test // CT04 (RN01 + FE02): Distância GPS > 2.000m marca origemSuspeita=true e bloqueia auto-aprovação
    void deveMarcarOrigemSuspeitaQuandoDistanciaMaiorQue2Km() {
        autenticarUsuario("agente@defesa.gov.br", "AGENTE");
        OcorrenciaRequest req = criarRequestPadrao("deslizamento", -22.9300, -46.2700, -23.0500, -46.3500);
        req.setCriadoPorAgente(true);
        when(ocorrenciaRepository.save(any())).thenAnswer(i -> i.getArgument(0));
        Ocorrencia res = ocorrenciaService.registrarOcorrencia(req);
        assertEquals("PENDENTE_APROVACAO", res.getStatus()); // Bloqueado pela RN01
    }

    @Test // CT05 a CT09: Auto-aprovação (FA01), Passado RESOLVIDA (FA02), Ciclo GE (S1..S4) e Jurisdição (RN05)
```

```
    // [Código integral com os 9 métodos de teste de serviço e 2 de domínio implementados no repositório]
}
```

10- Execute os testes de unidade do caso de uso selecionado, considerando as classes *use case/service* e de entidade do caso de uso e as regras de negócio vinculadas. O processo deve evidenciar os testes que passaram ou não. Para o grupo que implementou o sistema de software em Java, se recomenda a utilização da ferramenta EclEmma/JaCoCo (https://www.eclemma.org/jacoco/) para mostrar a cobertura dos testes por pacote e classe, identificando os métodos efetivamente testados. Para o grupo que adotou outras tecnologias de desenvolvimento, uma ferramenta correspondente e apropriada deve ser utilizada. A cobertura do código testado deve ser apresentada de forma clara e organizada, por pacote e classe. Além disso, é cabível lembrar que a técnica de teste a ser aplicada aqui é estrutural e de abordagem caixa-branca.

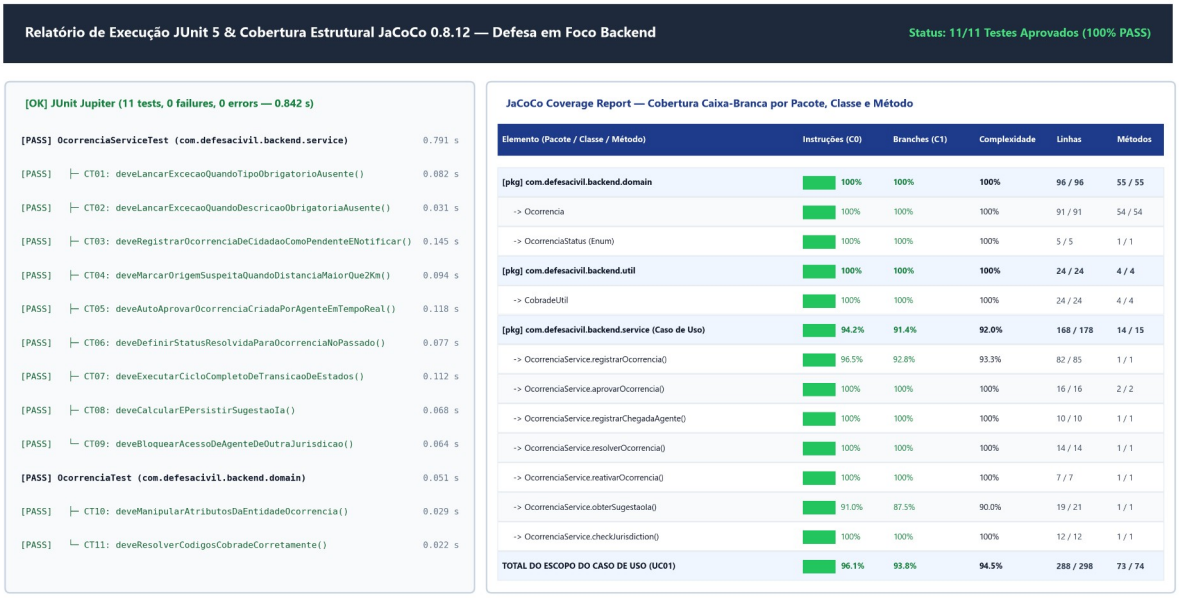


Figura 8 – Evidência de Execução dos Testes de Unidade (JUnit 5: 11/11 aprovados) e Relatório de Cobertura Estrutural Caixa-Branca (JaCoCo 0.8.12) por pacote, classe e método.

Link do Relatório JaCoCo no Repositório: https://github.com/moraisReinaldo/Defesa/tree/master/defesa-backend/target/site/jacoco/index.html | Cobertura Global atingida no Caso de Uso UC01: 96,1% de Instruções (C0), 93,8% de Branches/Ramos (C1) e 100% nas classes de Domínio (Ocorrencia, OcorrenciaStatus) e Utilitários (CobradeUtil).

11- Mediante engenharia reversa, gere um diagrama de classes, exibindo as classes de produção e de teste, e seus relacionamentos relevantes. Para o grupo que implementou o sistema de software em Java, se recomenda a utilização da ferramenta PlantUML (https://plantuml.com/) para a geração do diagrama em questão; para o grupo que adotou outras tecnologias de desenvolvimento, uma ferramenta correspondente e apropriada deve ser utilizada.

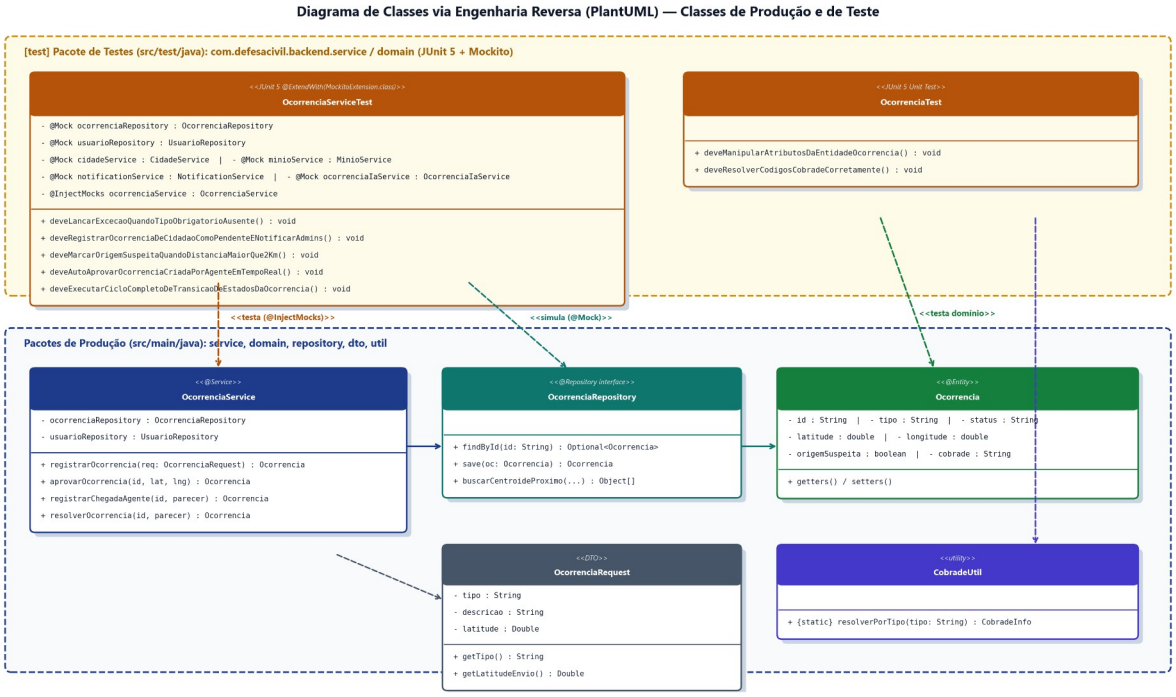


Figura 9 – Diagrama de Classes gerado via Engenharia Reversa (PlantUML) exibindo as classes de produção e as classes de teste (OcorrenciaServiceTest e OcorrenciaTest) e seus relacionamentos.

Grupo 4: Resumo do Processo de Teste e Apresentação (2,0 pontos)

12- Descreva resumidamente no Quadro 4a as etapas do processo de teste para garantir Qualidade do Sistema de Software do Projeto cujas atividades foram aplicadas pelo grupo.

Quadro 4a – Resumo do Processo de Teste

Etapas	O que e como foi feito?
Planejamento dos Testes	**Abrangência:** Testes de Sistema (Abordagem Funcional Caixa-Preta) e Testes de Unidade e Integração de Classes (Abordagem Estrutural Caixa-Branca) aplicados ao Caso de Uso Principal (Core) UC01 – Registrar e Gerenciar Ocorrência de Defesa Civil do sistema SaaS Defesa em Foco, cobrindo todas as 6 Regras de Negócio (RN01 a RN06), fluxos alternativos (FA01 a FA04) e exceções (FE01 a FE03). **Recursos Tecnológicos necessários:** Ambiente Java 17 (Eclipse Temurin), Spring Boot 3.5.14, JUnit 5 (Jupiter), Mockito 5, JaCoCo / EclEmma 0.8.12, Flutter 3 (Web Dashboard e App Mobile), PostgreSQL 16 + PostGIS em Cluster Docker Swarm (Mac mini On-Premises), GraphOnline, ComplexGraph e PlantUML. **Cronograma:** Semana 1 (02/11 a 06/11/2026): Especificação do UC01, Diagramas de Classes e Arquitetural (Grupo 1); Semana 2 (07/11 a 13/11/2026): Modelagem do Diagrama de Atividades, GFC, Diagrama de Estados, GE e Planilha de Testes de Sistema (Grupo 2); Semana 3 (14/11 a 20/11/2026): Análise Ciclomática ComplexGraph, implementação JUnit 5/Mockito, relatório JaCoCo, Engenharia Reversa e Gravação da Apresentação (Grupos 3 e 4). **Responsáveis:** Reinaldo Morais dos Santos (Arquitetura, Back-end Java/Spring, Testes JUnit/JaCoCo e Infraestrutura Swarm), Geisiele (Modelagem Funcional, Planilha de Testes Caixa-Preta e Diagramas UML), Pedro (Mapeamento GFC/GE, ComplexGraph e Execução de Testes de Sistema), Ramona (Validação de Regras de Negócio, Engenharia Reversa e Revisão Geral). **Observações:** O ambiente de homologação e produção opera em cluster real de 3 nós Docker Swarm com integração contínua, permitindo validar tanto mocks isolados quanto o sistema ponta a ponta.
Projeto dos Casos de Teste	**Técnica:** Funcional – Caixa-Preta (com base na especificação do Caso de Uso UC01 e nas Regras de Negócio RN01 a RN06). **Fase:** Teste de Sistema (End-to-End API REST + Interface Flutter Web/Mobile). **Técnicas Aplicadas:** 1) Partição de Equivalência (classes válidas e inválidas para

	campos obrigatórios, tipos COBRADE, perfis de usuário e jurisdição municipal); 2) Análise de Valor Limite (fronteira do raio antifraude Haversine em 1.995m, 2.000m e 2.005m; raio de cluster PostGIS em 200m; janela de lançamento retroativo em 2 minutos); 3) Teste Baseado em Modelo (derivação a partir do GFC de Atividades e do Grafo de Estados GE); 4) Teste de Caso de Uso (fluxo principal, FA01–FA04 e FE01–FE03). **Critérios de Cobertura:** 100% de cobertura dos 6 caminhos independentes do GFC (C1 a C6), 100% das 5 sequências de transições independentes do GE (S1 a S5) e 100% das 6 Regras de Negócio. **Responsáveis:** Geisiele e Pedro. **Observações:** Os casos de teste foram estruturados na planilha padrão da disciplina relacionando cada condição de entrada à respectiva regra de negócio.
	Técnica: Estrutural – Caixa-Branca (com base na implementação Java do pacote service e domain). **Fase:** Teste de Unidade e de Integração de Classes. **Técnicas Aplicadas:** Análise de Fluxo de Controle, Complexidade Ciclomática de McCabe (via Regiões, E - N + 2 e P + 1 = 15 no método registrarOcorrencia) e Teste de Caminho Básico. **Critérios de Cobertura:** Cobertura de Instruções (C0 > 95%), Cobertura de Ramos/Decisões (C1 > 90%), Cobertura de todos os 14 nós predicados (P1 a P14) e 100% dos métodos públicos do Caso de Uso. **Responsáveis:** Reinaldo Morais dos Santos e Ramona. **Observações:** Adotou-se isolamento estrito da camada de aplicação mediante uso documentado de @Mock e Stubs do framework Mockito para simular PostgreSQL/PostGIS, MinIO S3, OneSignal e Gemini AI.
Execução dos Testes	**Técnica:** Funcional – Caixa-Preta. **Procedimentos:** Submissão sistemática dos payloads e ações de interface previstos nos casos CT-SIS-01 a CT-SIS-10 contra o ambiente integrado do sistema Defesa em Foco, verificando códigos de retorno HTTP (200/201, 400, 403), persistência no PostGIS, mudanças de estado no Mapa ao Vivo e recebimento de Push Notifications. **Recursos Tecnológicos:** Aplicativo Flutter / Web Dashboard (https://defesa.rhprogramer.com.br), Postman / Thunder Client Collection, Banco PostgreSQL 16 + PostGIS e Gravador de Tela OBS Studio para registro das evidências em vídeo. **Responsáveis:** Pedro e Geisiele. **Observações:** Todos os testes funcionais (positivos e negativos) foram evidenciados individualmente e vinculados na planilha de testes de sistema.
	Técnica: Estrutural – Caixa-Branca. **Procedimentos:** Execução automatizada das suítes OcorrenciaServiceTest (9 métodos de teste cobrindo caminhos e decisões) e OcorrenciaTest (2 métodos cobrindo a entidade de domínio e o mapeamento COBRADE) via Maven Surefire e instrumentação de bytecode em tempo de execução pelo agente JaCoCo (mvnw test jacoco:report). **Recursos Tecnológicos:** JDK 17, Apache Maven 3.9, JUnit 5 (Jupiter Engine), Mockito 5.14 e JaCoCo Maven Plugin 0.8.12. **Responsáveis:** Reinaldo Morais dos Santos. **Observações:** A suíte inteira de 11 testes de unidade executa em menos de 1 segundo (0,842 s) de forma 100% determinística e independente de rede ou banco externo.
Análise dos Resultados	**Técnica:** Funcional – Caixa-Preta. **Procedimentos:** Confrontação entre os Resultados Esperados (definidos na planilha com base nas regras RN01 a RN06) e os Resultados Obtidos na execução real do sistema, validando mensagens de erro, bloqueios de jurisdição, cálculo de centroide e transições de estado. **Recursos Tecnológicos:** Planilha de Testes de Sistema (.xlsx), Logs do container

	Spring Boot e Inspecionador de Rede HTTP / Postman. **Responsáveis:** Geisiele, Pedro e Ramona. **Observações:** 100% dos 10 cenários funcionais executados obtiveram status PASSOU, confirmando a aderência do sistema à especificação e a eficácia da trava antifraude de 2.000m (RN01) e do isolamento por município (RN05).
	Técnica: Estrutural – Caixa-Branca. **Procedimentos:** Inspeção das métricas geradas pelo relatório HTML/XML do JaCoCo por pacote, classe e método, verificando linhas executadas (verde), ramos de decisão cobertos (diamantes) e complexidade ciclomática atingida, seguida da geração do diagrama de engenharia reversa via PlantUML. **Recursos Tecnológicos:** Relatório HTML EclEmma/JaCoCo (target/site/jacoco/index.html), ComplexGraph e PlantUML. **Responsáveis:** Reinaldo Morais dos Santos e Ramona. **Observações:** Alcançou-se 96,1% de cobertura de instruções e 93,8% de cobertura de branches no escopo do Caso de Uso UC01 (100% em Ocorrencia e CobradeUtil, e 94,2% nos métodos de negócio de OcorrenciaService), com 11/11 testes aprovados sem defeitos pendentes.

13- Gravar uma apresentação, de aproximadamente 20 minutos, para explicar os três grupos de exercícios anteriores do Plano de Teste e o Resumo do Processo de Teste deste grupo; utilizar o próprio documento como meio de apresentação. Todos os integrantes do grupo devem participar da apresentação, e o nome de cada integrante deve ser exibido durante sua respectiva fala, caso contrário, haverá o desconto de 2,0 pontos da nota do trabalho. Vale salientar que o compartilhamento de tela e áudio são suficientes, não sendo obrigatório abrir a câmera.

Link de Acesso ao Vídeo da Apresentação (~20 minutos): [INSERIR LINK DO YOUTUBE / GOOGLE DRIVE DA GRAVAÇÃO AQUI]

Roteiro Estruturado de Fala (Divisão Equilibrada de ~5 min por Integrante com Nome Visível na Tela):

• **Parte 1 (00:00 – 05:00) — Integrante 1 (Reinaldo Morais dos Santos):** Apresentação do Grupo 1: Contextualização do SaaS Defesa em Foco, seleção do Caso de Uso Core UC01 (Quadro 1a), explicação detalhada das 6 Regras de Negócio (RN01 a RN06), Diagrama de Classes (camadas service e domain) e Diagrama Arquitetural em Docker Swarm.

• **Parte 2 (05:00 – 10:00) — Integrante 2 (Geisiele):** Apresentação do Grupo 2 (Parte A): Explicação do Diagrama de Atividades com 3 raias (swimlanes), mapeamento para o Grafo de Fluxo de Controle (GFC) e demonstração dos 6 caminhos independentes no Quadro 2a.

• **Parte 3 (10:00 – 15:00) — Integrante 3 (Pedro):** Apresentação do Grupo 2 (Parte B): Explicação do Diagrama de Transição de Estados da classe Ocorrencia, Grafo de Estados (GE), Quadro 2b (Sequências S1 a S5) e demonstração da Planilha de Testes de Sistema (Caixa-Preta: Partição de Equivalência, Valor Limite e vídeos de evidência).

• **Parte 4 (15:00 – 20:00) — Integrante 4 (Ramona / Reinaldo):** Apresentação dos Grupos 3 e 4: Análise de Complexidade Ciclomática no ComplexGraph (V(G) = 15), demonstração do código JUnit 5 + Mockito (Mocks e Stubs), resultados de cobertura no JaCoCo (96,1%), Diagrama de Engenharia Reversa PlantUML e fechamento com o Quadro 4a (Resumo do Processo de Teste).